



Aqueduct Materials (aqueducts)

Les aqueducs sont construits avec l'un des N matériaux romains. Le i -ème matériau a une solidité S_i et un prix P_i .

On dit que le i -ème matériau *domine* le j -ème matériau si et seulement si :

- il est plus solide ($S_i > S_j$), et
- il est moins cher ($P_i < P_j$).

Le Sénat veut restreindre l'usage d'un matériau exclusivement à Rome afin que la cité maintienne une apparence unique. Les $N - 1$ autres matériaux seront *disponibles* pour le reste de l'empire.

Un matériau est *optimal* pour le reste de l'empire si

- il est disponible, et
- il n'est dominé par aucun autre matériau disponible.

Pour chaque k , en supposant que le matériau réservé à Rome soit k , comptez le nombre C_k de matériaux optimaux pour le reste de l'empire.

Implémentation

Vous devez soumettre un seul fichier source `.cpp`.



Parmi les pièces jointes du problème, vous trouverez un squelette `aqueducts.cpp` avec une implémentation d'exemple.

Vous devez implémenter la fonction suivante :

C++

```
vector<int> count(int N, vector<int> S, vector<int> P)
```

- L'entier N représente le nombre de matériaux.
- Le tableau S , indexé de 0 à $N - 1$, contient les valeurs $S_0, S_1, \dots, S_{N-1}$.
- Le tableau P , indexé de 0 à $N - 1$, contient les valeurs $P_0, P_1, \dots, P_{N-1}$.
- La fonction doit renvoyer un tableau C , contenant les valeurs $C_0, C_1, \dots, C_{N-1}$ où C_k est le nombre de matériaux optimaux pour le reste de l'empire si k est réservé à Rome.

Évaluateur d'exemple

Parmi les fichiers joints à cet énoncé, vous trouverez une version simplifiée du grader utilisé lors de l'évaluation. Vous pouvez l'utiliser pour tester vos solutions en local. Ce grader lit les données depuis `stdin`, appelle la fonction `count` et écrit le résultat dans `stdout` selon le format suivant.

L'entrée est composée de $N + 1$ lignes, contenant :

- Ligne 1 : l'entier N .
- Ligne $2 + i$ ($0 \leq i < N$) : les entiers S_i et P_i .

La sortie est composée d'une seule ligne contenant les valeurs retournées par la fonction `count`.

Contraintes

- $2 \leq N \leq 300\,000$.
- $1 \leq S_i, P_i \leq 1\,000\,000$.
- Pour tous $0 \leq i < j \leq N - 1$, on a que $S_i \neq S_j$ et $P_i \neq P_j$.

Score

Votre programme sera testé sur plusieurs tests, regroupés en sous-tâches. Pour obtenir les points d'une sous-tâche, votre programme doit résoudre correctement tous les tests qui lui sont associés.

- **Sous-tâche 0 [0 points]**: Exemples.
- **Sous-tâche 1 [13 points]**: $N \leq 100$.
- **Sous-tâche 2 [22 points]**: $N \leq 5000$.
- **Sous-tâche 3 [28 points]**: $|S_i - P_i| \leq 2$.
- **Sous-tâche 4 [37 points]**: Pas de contraintes additionnelles.

Exemples

stdin	stdout
3 10 20 30 10 20 30	1 2 1
9 10 11 4 6 11 13 14 7 1 4 6 10 9 2 12 12 3 5	2 2 2 3 2 2 4 2 2

Explication

Explication du premier exemple.

- Si $k = 0$, le matériau 1 est optimal.
- Si $k = 1$, les matériaux 0 et 2 sont optimaux.
- Si $k = 2$, le matériau 1 est optimal.

Explication du second exemple.

- Si $k = 0$, les matériaux 3 et 6 sont optimaux.
- Si $k = 1$, les matériaux 3 et 6 sont optimaux.
- Si $k = 2$, les matériaux 3 et 6 sont optimaux.
- Si $k = 3$, les matériaux 0, 6 et 7 sont optimaux.
- Si $k = 4$, les matériaux 3 et 6 sont optimaux.
- Si $k = 5$, les matériaux 3 et 6 sont optimaux.
- Si $k = 6$, les matériaux 1, 3, 4 et 8 sont optimaux.
- Si $k = 7$, les matériaux 3 et 6 sont optimaux.
- Si $k = 8$, les matériaux 3 et 6 sont optimaux.